MANIPAL SCHOOL OF INFORMATION SCIENCES

Manipal Academy of Higher Education

LINUX OS AND SCRIPTING

Assignment: Process Management, Signals and Scheduling

PROCESS MANAGEMENT AND AUTOMATION IN LINUX

Subject: Linux OS and Scripting

Programme: M.Tech Cyber Security

Department: Cyber Security (CYS)

Student Name: Shreyas S. Sanil

Roll Number: 261100690003

Academic Year: 2026–2027

Manipal Academy of Higher Education
Manipal, Karnataka

Contents

1 Introduction **1**

1.1 Objectives 1

2 Exercise 1: Process Identification **3**

2.1 Aim 3

2.2 Commands Used 3

2.3 Explanation 3

2.4 Why High CPU Usage Alone Does Not Confirm Malicious Activity 4

2.5 Result 5

3 Exercise 2: Parent and Child Processes **6**

3.1 Aim 6

3.2 Bash Program 6

3.3 Execution 7

3.4 Important Bash Variables and Operators 7

3.5 Parent and Child Process Identification 7

3.6 What Happens When the Parent Terminates? 7

3.7 Result 8

4 Exercise 3: Process Control Using kill **9**

4.1 Aim 9

4.2 Procedure 9

4.3 Explanation of Signals 10

4.4 Result 10

5 Exercise 4: Graceful and Forced Termination **11**

5.1 Aim . . . 11

5.2 Bash Program . . . 11

5.3 Testing SIGTERM . . . 12

5.4 Testing SIGKILL . . . 12

5.5 Comparison . . . 13

5.6 Result . . . 13

6 Exercise 5: Multiple Processes Using killall **14**

6.1 Aim . . . 14

6.2 Create a Lab Program . . . 14

6.3 Start Three Instances . . . 14

6.4 Terminate One Instance . . . 15

6.5 Terminate Remaining Instances . . . 15

6.6 Explanation . . . 15

6.7 Result . . . 15

7 Exercise 6: Recurring Process Monitoring Using cron **16**

7.1 Aim . . . 16

7.2 Bash Program . . . 16

7.3 Configure Cron . . . 16

7.4 Verify Execution . . . 17

7.5 Modify the Schedule . . . 17

7.6 Remove Only the Exercise Entry . . . 17

7.7 Result . . . 18

8 Exercise 7: Cron Scheduling **19**

8.1 Aim . . . 19

8.2 Cron Format . . . 19

8.3 Required Cron Entries . . . 19

8.4 Explanation . . . 20

8.5 Result 20

9 Exercise 8: One-Time Scheduling Using at 21

9.1 Aim 21

9.2 Schedule a Task 21

9.3 Display Pending Jobs 21

9.4 Inspect Scheduled Commands 21

9.5 Verify Execution 22

9.6 Explanation 22

9.7 Result 22

10 Exercise 9: Cancelling Scheduled Jobs 23

10.1 Aim 23

10.2 Schedule a Task 23

10.3 Identify the Job Number 23

10.4 Cancel the Job 23

10.5 Verify Cancellation 24

10.6 Difference Between at and cron 24

10.7 Result 24

11 Exercise 10: Incident-Response Scenario 25

11.1 Aim 25

11.2 Scenario 25

11.3 Step 1: Identify Process Instances 25

11.4 Step 2: Suspend One Instance 26

11.5 Step 3: Resume the Process 26

11.6 Step 4: Terminate All Lab Instances 26

11.7 Step 5: Inspect Scheduled Restart Tasks 26

11.8 Step 6: Cancel the Scheduled Restart 27

11.9 Step 7: Final Verification 27

11.10Incident Report 27

11.10.1 Incident Summary 27
11.10.2 Investigation Activities 28
11.10.3 Response Actions 28
11.10.4 Verification 28
11.11 Result 28

12 Overall Learning Outcomes **29**

13 Conclusion **30**

14 GitHub Repository **31**

Chapter 1

Introduction

Process management is an important function of a Linux operating system. A process is an instance of a running program. Linux provides several commands and system utilities to create, monitor, control, and terminate processes.

Every process is associated with a Process ID (PID), Parent Process ID (PPID), owner, and resource usage information. Linux also supports process communication, signals, and scheduling mechanisms.

Process management is particularly important in cybersecurity because suspicious processes may consume system resources, execute unauthorized commands, or attempt to maintain persistence through scheduled tasks.

This assignment explores process identification, parent-child relationships, process control using signals, process termination, cron scheduling, and one-time job scheduling using the `at` command.

1.1 Objectives

1. Identify running processes and their resource usage.
2. Understand parent and child process relationships.
3. Control processes using Linux signals.
4. Compare graceful and forced process termination.
5. Manage multiple process instances using `killall`.
6. Schedule recurring tasks using cron.
7. Understand cron scheduling expressions.

8. Schedule one-time tasks using the `at` command.
9. Cancel scheduled jobs.
10. Apply process management concepts to an incident-response scenario.

Chapter 2

Exercise 1: Process Identification

2.1 Aim

To display running processes, identify processes belonging to the current user, and determine the processes consuming the highest CPU and memory.

2.2 Commands Used

```
# Display all running processes
ps -eo pid,ppid,user,%cpu,%mem,cmd

# Display processes belonging to the current user
ps -u "$USER" -o pid,ppid,user,%cpu,%mem,cmd

# Display top five CPU-consuming processes
ps -eo pid,ppid,user,%cpu,%mem,cmd \
--sort=-%cpu | head -n 6

# Display top five memory-consuming processes
ps -eo pid,ppid,user,%cpu,%mem,cmd \
--sort=-%mem | head -n 6
```

2.3 Explanation

- `ps`: Displays information about processes.

- `-e`: Selects all processes.
- `-o`: Specifies the output columns.
- `PID`: Process identification number.
- `PPID`: Parent process identification number.
- `USER`: Process owner.
- `%CPU`: CPU utilization.
- `%MEM`: Memory utilization.
- `CMD`: Command associated with the process.
- `-sort=-%cpu`: Sorts processes by CPU usage in descending order.
- `-sort=-%mem`: Sorts processes by memory usage in descending order.

The `head -n 6` command displays the header and the five processes with the highest resource usage.

2.4 Why High CPU Usage Alone Does Not Confirm Malicious Activity

High CPU utilization does not automatically indicate malicious activity. Legitimate applications such as software compilation, video processing, database operations, and system updates can consume significant CPU resources.

To identify suspicious processes, an analyst should also examine:

- Process name and executable path.
- Process owner and parent process.
- Network connections.
- Execution history and associated files.
- Unexpected resource consumption patterns.

2.5 Result

Running processes can be identified using their PID, PPID, owner, CPU usage, memory usage, and command. Processes can also be filtered by user and sorted according to resource consumption.

Chapter 3

Exercise 2: Parent and Child Processes

3.1 Aim

To create background processes using a shell script and understand the relationship between parent and child processes.

3.2 Bash Program

Create a file named `parent_child.sh`.

```
#!/bin/bash

echo "Parent Script PID: $$"
echo "Parent PPID: $PPID"

sleep 100 &
PID1=$!

sleep 120 &
PID2=$!

echo "First Child PID: $PID1"
echo "Second Child PID: $PID2"

echo "Child processes:"
ps -o pid,ppid,stat,cmd -p "$PID1","$PID2"

wait
```

```
19 echo "Parent script completed."
```

3.3 Execution

```
1 chmod +x parent_child.sh
2 ./parent_child.sh
```

3.4 Important Bash Variables and Operators

Symbol	Purpose
`$$`	Stores the PID of the current shell or script process.
`$!`	Stores the PID of the most recently started background process.
`&`	Runs a command in the background.
`wait`	Waits for background child processes to complete.

Table 3.1: Bash process management symbols

3.5 Parent and Child Process Identification

The script creates two background processes using the `sleep` command.

The parent script PID can be identified using `$$`, while the child PIDs are stored using `$!` immediately after starting each process.

The following command can also be used to inspect the child processes:

```
1 ps --ppid <PARENT_PID> -o pid,ppid,stat,cmd
```

3.6 What Happens When the Parent Terminates?

When a parent process terminates, its child processes do not necessarily terminate automatically.

Depending on how the processes are managed, child processes may continue running and become orphan processes. They are then adopted by an appropriate system process, commonly PID 1 or a subreaper.

The `wait` command allows the parent shell to wait for its child processes to finish.

3.7 Result

Two background child processes were created and their relationship with the parent shell was examined using PID and PPID information.

Chapter 4

Exercise 3: Process Control Using kill

4.1 Aim

To suspend, resume, and terminate a background process using Linux signals.

4.2 Procedure

Start a background process:

```
sleep 300 &
PID=$!

echo "Process PID: $PID"
```

Suspend the process:

```
kill -STOP "$PID"
ps -o pid,stat,cmd -p "$PID"
```

Resume the process:

```
kill -CONT "$PID"
ps -o pid,stat,cmd -p "$PID"
```

Terminate the process gracefully:

```
kill -TERM "$PID"
```

Verify termination:

```
ps -p "$PID"
```

If the process has terminated, it will no longer appear in the process listing.

4.3 Explanation of Signals

Signal	Meaning	Purpose
SIGSTOP	Stop process	Suspends a process and cannot be caught or ignored.
SIGCONT	Continue process	Resumes a stopped process.
SIGTERM	Terminate process	Requests graceful termination and can be handled by the process.
SIGKILL	Kill process	Forces termination and cannot be caught or ignored.

Table 4.1: Common process control signals

4.4 Result

The background process was suspended, resumed, and terminated using Linux signals. Its state was verified using the `ps` command.

Chapter 5

Exercise 4: Graceful and Forced Termination

5.1 Aim

To create a monitoring script that records timestamps and handles SIGTERM gracefully, and compare its behaviour with SIGKILL.

5.2 Bash Program

Create a file named `monitor.sh`.

```
#!/bin/bash

LOGFILE="monitor.log"

echo "Monitoring started: $(date)" >> "$LOGFILE"

cleanup() {
    echo "Shutdown received: $(date)" >> "$LOGFILE"
    exit 0
}

trap cleanup SIGTERM

while true
do
    echo "Timestamp: $(date)" >> "$LOGFILE"
```

```
    sleep 5
done
```

5.3 Testing SIGTERM

Start the script:

```
chmod +x monitor.sh
./monitor.sh &
PID=$!

echo "Monitoring PID: $PID"
```

Terminate gracefully:

```
kill -TERM "$PID"
```

Inspect the log:

```
cat monitor.log
```

The script receives SIGTERM, executes the cleanup function, records a shutdown message, and exits.

5.4 Testing SIGKILL

Restart the monitoring script:

```
./monitor.sh &
PID=$!
```

Forcefully terminate it:

```
kill -KILL "$PID"
```

Inspect the log again:

```
cat monitor.log
```

5.5 Comparison

Feature	**SIGTERM**	**SIGKILL**
Termination	Graceful request	Immediate forced termination
Signal handling	Can be caught and handled	Cannot be caught or ignored
Cleanup	Cleanup code can execute	Cleanup code cannot execute
Log message	Shutdown message can be recorded	Shutdown message is not recorded by the killed process

Table 5.1: SIGTERM and SIGKILL comparison

5.6 Result

The monitoring script records timestamps in a log file. SIGTERM allows the script to execute its shutdown handler, whereas SIGKILL terminates it without executing cleanup code.

Chapter 6

Exercise 5: Multiple Processes Using killall

6.1 Aim

To start multiple instances of a lab program, terminate one instance using its PID, and terminate the remaining instances using `killall`.

6.2 Create a Lab Program

Create a script named `labworker`.

```
#!/bin/bash

exec -a labworker sleep 600
```

`exec -a` sets the process argument name for the executed command. The actual executable remains `sleep`.

6.3 Start Three Instances

```
chmod +x labworker

./labworker &
./labworker &
./labworker &
```

Identify the processes:

```
pgrep -af labworker

ps -eo pid,user,args | grep '[l]abworker'
```

6.4 Terminate One Instance

Identify a PID from the process listing and terminate that process:

```
kill <PID>
```

6.5 Terminate Remaining Instances

Use the following command:

```
killall labworker
```

Verify:

```
pgrep -af labworker
```

6.6 Explanation

The `kill` command sends a signal to a specific process using its PID.

The `killall` command sends a signal to processes matching a specified process name.

Using `killall` with a common process name can accidentally terminate legitimate system or application processes.

Therefore, process names should be verified before using broad termination commands.

Note: The process-name matching behaviour of `killall` depends on the Linux implementation. The lab example should be tested in the target environment.

6.7 Result

Multiple lab process instances were created and identified. One instance was terminated using its PID, and the remaining instances were targeted using `killall`.

Chapter 7

Exercise 6: Recurring Process Monitoring Using cron

7.1 Aim

To create a shell script that records the current date, time, username, and number of processes belonging to the current user, and schedule it to execute every minute.

7.2 Bash Program

Create a file named `process_monitor.sh`.

```
#!/bin/bash

LOGFILE="$HOME/process_monitor.log"

DATE_TIME=$(date '+%Y-%m-%d %H:%M:%S')
USERNAME=$(whoami)
PROCESS_COUNT=$(ps -u "$USERNAME" --no-headers | wc -l)

echo "$DATE_TIME | User: $USERNAME | Processes: $PROCESS_COUNT" \
>> "$LOGFILE"
```

7.3 Configure Cron

Make the script executable:

```
chmod +x ~/process_monitor.sh
```

Open the current user's crontab:

```
crontab -e
```

Add the following entry:

```
* * * * * /bin/bash /home/username/process_monitor.sh
```

Replace `username` with the actual Linux username.

7.4 Verify Execution

Wait for the next scheduled minute and inspect:

```
cat ~/process_monitor.log
```

Start additional background processes:

```
sleep 300 &
sleep 400 &
```

Observe whether the process count changes in subsequent log entries.

7.5 Modify the Schedule

Edit the crontab:

```
crontab -e
```

Change the entry to:

```
*/5 * * * * /bin/bash /home/username/process_monitor.sh
```

The script will now execute every five minutes.

7.6 Remove Only the Exercise Entry

Open the crontab again and remove only the process monitoring entry.

```
crontab -e
```

Other cron entries should remain unchanged.

7.7 Result

A recurring process monitoring script was created and scheduled using cron. The log records the username, timestamp, and process count.

Chapter 8

Exercise 7: Cron Scheduling

8.1 Aim

To write cron expressions for different recurring security monitoring and maintenance activities.

8.2 Cron Format

A cron entry contains five time fields followed by the command to execute.

```
* * * * * command
| | | | |
| | | | +---- Day of week
| | | +------ Month
| | +-------- Day of month
| +---------- Hour
+------------ Minute
```

8.3 Required Cron Entries

Listing 8.1: Security-related cron schedules

```
# Check failed-login logs every five minutes
*/5 * * * * /bin/bash /home/username/check_failed_logins.sh

# Capture a process snapshot at the beginning of every hour
0 * * * * /bin/bash /home/username/process_snapshot.sh

```

```
# Back up security logs daily at 2:30 AM
30 2 * * * /bin/bash /home/username/backup_security_logs.sh

# Generate a security report every weekday at 9:00 AM
0 9 * * 1-5 /bin/bash /home/username/security_report.sh

# Perform file-integrity check every Sunday at 3:00 AM
0 3 * * 0 /bin/bash /home/username/file_integrity.sh

# Run a monitoring script every 15 minutes
*/15 * * * * /bin/bash /home/username/monitor.sh

# Generate a weekly report every Monday at 8:30 AM
30 8 * * 1 /bin/bash /home/username/weekly_report.sh
```

8.4 Explanation

Task	Cron Expression	Schedule
Failed-login check	`*/5 * * * *`	Every five minutes
Process snapshot	`0 * * * *`	Every hour at minute zero
Daily backup	`30 2 * * *`	Every day at 2:30 AM
Weekday report	`0 9 * * 1-5`	Monday to Friday at 9 AM
Integrity check	`0 3 * * 0`	Sunday at 3 AM
Monitoring	`*/15 * * * *`	Every 15 minutes
Weekly report	`30 8 * * 1`	Monday at 8:30 AM

Table 8.1: Cron scheduling requirements

8.5 Result

Cron expressions were prepared for recurring security monitoring, process snapshots, backups, integrity checks, and report generation.

Chapter 9

Exercise 8: One-Time Scheduling Using at

9.1 Aim

To schedule a one-time task using the Linux `at` command and verify its execution.

9.2 Schedule a Task

Schedule a task to execute two minutes later:

```
at now + 2 minutes
```

Enter the following commands at the `at` prompt:

```
date > "$HOME/at_report.txt"
ps -ef >> "$HOME/at_report.txt"
```

Press `Ctrl+D` to submit the job.

9.3 Display Pending Jobs

```
atq
```

9.4 Inspect Scheduled Commands

```
at -c <JOB_NUMBER>
```

The job number should be replaced with the number returned by `atq`.

9.5 Verify Execution

After the scheduled time, inspect the output file:

```
cat "$HOME/at_report.txt"
```

9.6 Explanation

The `at` command schedules a task to run once at a specified future time.

The `atq` command lists pending jobs. After a job executes successfully, it is normally removed from the pending-job list.

The `at -c` command displays the commands associated with a scheduled job.

9.7 Result

A one-time task was scheduled to record the current date and process list. Its output was verified after execution.

Chapter 10

Exercise 9: Cancelling Scheduled Jobs

10.1 Aim

To schedule a harmless one-time task, identify its job number, and cancel it before execution.

10.2 Schedule a Task

```
at now + 10 minutes
```

Enter:

```
echo "Scheduled lab task executed at $(date)" \
>> "$HOME/at_cancel_test.log"
```

Press `Ctrl+D` to submit the job.

10.3 Identify the Job Number

```
atq
```

10.4 Cancel the Job

```
atrm <JOB_NUMBER>
```

10.5 Verify Cancellation

```
atq
```

The cancelled job should no longer appear in the pending-job list.

10.6 Difference Between at and cron

- `at` is used for one-time scheduled tasks.
- `atrm` removes a specific pending `at` job.
- `cron` is used for recurring scheduled tasks.
- Removing a cron entry requires editing the user's crontab.
- Removing one cron entry does not require deleting other entries.

10.7 Result

A one-time scheduled task was identified and cancelled using its job number. The remaining scheduled jobs were preserved.

Chapter 11

Exercise 10: Incident-Response Scenario

11.1 Aim

To investigate multiple instances of an unfamiliar lab process and identify and cancel a scheduled task that may restart it.

11.2 Scenario

An unfamiliar lab process has multiple running instances and a scheduled task that will restart it.

The objective is to identify the instances, inspect their owners and process IDs, control their execution, terminate them, inspect the scheduled restart task, and verify that no instances or restart jobs remain.

11.3 Step 1: Identify Process Instances

```
pgrep -af labworker

ps -eo pid,ppid,user,stat,cmd | grep '[l]abworker'
```

The commands display matching process IDs, parent process IDs, owners, states, and commands.

11.4 Step 2: Suspend One Instance

Select one PID from the process list:

```
kill -STOP <PID>
```

Verify its state:

```
ps -o pid,ppid,user,stat,cmd -p <PID>
```

A stopped process is commonly indicated by the state character T.

11.5 Step 3: Resume the Process

```
kill -CONT <PID>
```

Verify that the process is no longer in the stopped state.

11.6 Step 4: Terminate All Lab Instances

After confirming the process name and scope, terminate the lab instances:

```
killall labworker
```

Verify:

```
pgrep -af labworker
```

No matching process should remain.

11.7 Step 5: Inspect Scheduled Restart Tasks

Inspect the current user's cron entries:

```
crontab -l
```

Inspect system-wide cron locations where the user has appropriate permissions:

```
ls -l /etc/cron.d/
ls -l /etc/cron.hourly/
ls -l /etc/cron.daily/
```

Search for a scheduled command that launches the lab process:

```
grep -R "labworker" /etc/cron.d/ /etc/crontab 2>/dev/null
```

11.8 Step 6: Cancel the Scheduled Restart

If the restart task is in the current user's crontab, edit it:

```
crontab -e
```

Remove only the verified unauthorized restart entry. Preserve unrelated scheduled tasks.

If the task is an `at` job, inspect pending jobs and remove the identified job:

```
atq
atrm <JOB_NUMBER>
```

System-wide scheduled tasks should only be changed by an authorized administrator.

11.9 Step 7: Final Verification

Check for remaining process instances:

```
pgrep -af labworker
```

Check the current user's cron entries:

```
crontab -l
```

Check pending one-time jobs:

```
atq
```

Verify that the identified restart entry has been removed and that no lab process instances remain.

11.10 Incident Report

11.10.1 Incident Summary

An unfamiliar lab process was observed running with multiple instances. A scheduled task was also identified that could restart the process.

11.10.2 Investigation Activities

The investigation involved identifying process instances, recording their PIDs and owners, examining parent-child relationships, and checking the process state.

One process was suspended and subsequently resumed to observe the effect of process control signals.

11.10.3 Response Actions

The lab process instances were terminated after their identity and scope were verified.

The scheduled restart task was inspected and the identified entry was removed from its corresponding scheduling mechanism.

11.10.4 Verification

Final verification involved checking the process list, current user's cron entries, and pending one-time scheduled jobs.

Observation: Actual PIDs, usernames, timestamps, and command outputs should be recorded from the practical execution environment.

11.11 Result

The incident-response procedure demonstrates how Linux process management and scheduling tools can be used to investigate unfamiliar processes, control their execution, remove identified restart tasks, and verify the final system state.

Chapter 12

Overall Learning Outcomes

After completing the ten exercises, the following concepts and practical skills were understood:

1. Identifying processes using PID, PPID, owner, CPU usage, and memory usage.
2. Understanding parent and child process relationships.
3. Creating and monitoring background processes.
4. Using SIGSTOP, SIGCONT, SIGTERM, and SIGKILL for process control.
5. Understanding graceful and forced termination.
6. Managing multiple process instances.
7. Automating recurring monitoring tasks using cron.
8. Writing cron expressions for security activities.
9. Scheduling and cancelling one-time tasks using `at` and `atrm`.
10. Applying process and scheduling analysis to a basic incident-response scenario.

Chapter 13

Conclusion

This assignment provided practical knowledge of Linux process management, process control, and task scheduling.

The exercises demonstrated how to identify running processes, inspect their resource consumption, understand parent-child relationships, and control process execution using Linux signals.

The assignment also explored graceful and forced termination, management of multiple process instances, recurring task automation using cron, and one-time scheduling using the `at` command.

The incident-response exercise combined these concepts to demonstrate how unfamiliar processes and scheduled restart mechanisms can be investigated and controlled.

These techniques are important in cybersecurity because monitoring processes and scheduled tasks can help identify suspicious activities and unauthorized persistence mechanisms.

Overall, the assignment strengthened practical understanding of Linux system administration, process monitoring, automation, and basic incident-response procedures.

Chapter 14

GitHub Repository

The practical scripts and related Linux OS and Scripting assignment materials are maintained in the following GitHub repository.

Linux OS and Scripting – Assignments

https://github.com/Shreyas-2004-ai/LOS-Assignments

Click here to view the GitHub repository